

Chapitre 1

Systeme et ligne de commande

Réponse exercice 1.1. -k

Réponse exercice 1.2.

1. /home/eleve/Info/dm1
2. /usr/include
3. include/math.h
4. ../../bin/firefox

Réponse exercice 1.3.

1. cd /
2. cd ..
cd

Réponse exercice 1.4. 1. cp dm1 ../../prof/MP2I

2. rm /bin/libreoffice

Chapitre 2

Fichiers

Réponse exercice 2.1.

1. tout le monde
2. root, eleve2, eleve3
3. tout le monde
4. root, prof

Réponse exercice 2.2. 1, 5, 6, 8, 9

Chapitre 3

Caractériser un langage de programmation

Le langage C : les bases

Réponse exercice 4.2. En numérotant les apparitions :

Réponse exercice 4.3. `int valeur_absolue(double x);`

Réponse exercice 4.5. 00101010

Réponse exercice 4.6. $-17 : 11101111$ $-42 : 11010110$ $-80 : 10110000$

Réponse exercice 4.7.

[illegible]

Chapitre 5

Introduction à l'algorithmique

Chapitre 6

Correction totale

Réponse exercice 6.1. Il y a une infinité de réponses possibles pour le variant : n est un variant de boucle, $n + 42$ aussi, ainsi que $n - 17$.

Réponse exercice 6.2. La boucle possède un variant, donc un nombre fini d'itérations.

Comme toutes les instructions dans la boucle se terminent, l'exécution de la boucle se termine. La fonction `mystere` ne contient que la boucle et le retour, tout appel à `mystere` se termine aussi.

Réponse exercice 6.3. Il y a une infinité de réponses : n convient, mais également $n + 42$ par exemple.

Réponse exercice 6.4. L'existence d'un variant d'appel assure qu'il y a un nombre fini d'appels récursifs. Les autres instructions se terminent car ce sont des opérations entre entiers (modulos, produits, tests d'égalité), donc tout appel à `puissance` se termine.

Réponse exercice 6.5. C'est clairement vrai avant la boucle. Supposons que ce soit vrai au début d'une itération et notons m et n les valeurs respectives de `m` et `n` au début de cette itération et m' et n' leurs valeurs respectives à la fin de cette itération. On a alors : $m + n = m_0 + n_0$ par hypothèse, et $m' = m + 1$ et $n' = n - 1$ suite aux lignes 4 et 5, donc la propriété est vraie en fin d'itération car $m' + n' = m + 1 + n - 1 = m + n = m_0 + n_0$.

En sortie de boucle, les valeurs finales $m + f$ et n_f de `m` et `n` vérifient donc $m_f + n_f = m_0 + n_0$. Or en sortie de boucle on a $n_f = 0$, donc $m_f = m_0 + n_0$, ce qui nous assure la correction de la fonction. *Attention : on n'a pas prouvé la correction totale, il faudrait montrer que tout appel se termine également pour cela.*

Réponse exercice 6.6. Montrons que le calcul est correct pour tout appel à `somme`, par récurrence sur $n \geq 0$.

cas de base : $n = 0$. Alors $m + n = m$ et le résultat est renvoyé en ligne 4. Le calcul est donc correct.

hérédité : Supposons que tout appel à `somme` se termine et est correct pour une certaine valeur $n \in \mathbb{N}$ de `n`. Soit un appel à `somme` pour un certain m et $n + 1$. L'appel récursif de la ligne 6 se termine et est correct par hypothèse de récurrence : il renvoie $m + 1 + n - 1 = m + n$, donc l'appel courant se termine et est correct.

Chapitre 7

Le langage C : gestion de la mémoire

Réponse exercice 7.1.

```
int tab[100];
for(int i=0; i < 100; i=i+1){
    tab[i] = 2*i;
}
```

Réponse exercice 7.2.

```
int F[100] = {0, 1};
for(int i=2; i < 100; i=i+1){
    F[i] = F[i-1] + F[i-2];
}
```

Réponse exercice 7.3.

```
void translation(int *t, int n, int v){
    for(int i=0; i<n; i=i+1){
        t[i] = t[i] + v;
    }
}
```

Réponse exercice 7.4.

```
int *int_N(int n){
    int *res = malloc(n * sizeof(int));
    for(int i=0; i<n; i=i+1){
        res[i] = i;
    }
    return res;
}
```

Réponse exercice 7.5.

```
int main(int argc, char **argv){
    if(argc != 2){
        return 1;
    }
    printf("%d\n", atoi(argv[1]) + atoi(argv[2]));
    return 0;
}
```